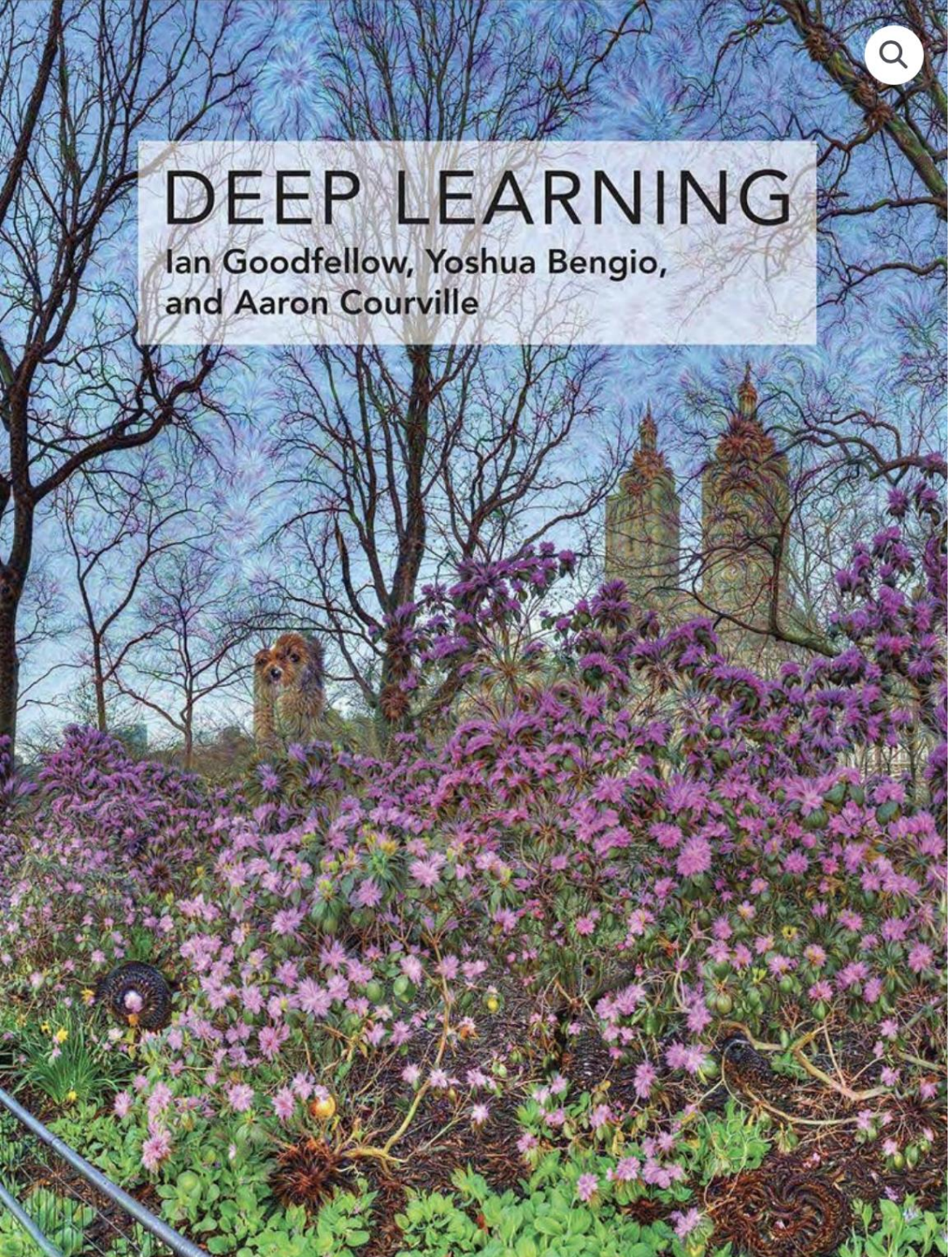
Deep Learning Systems 2024:
Lecture 4 – Brains to Backprop Part 1



Crescat scientia; vita excolatur

Rick Stevens
Argonne National Laboratory
The University of Chicago



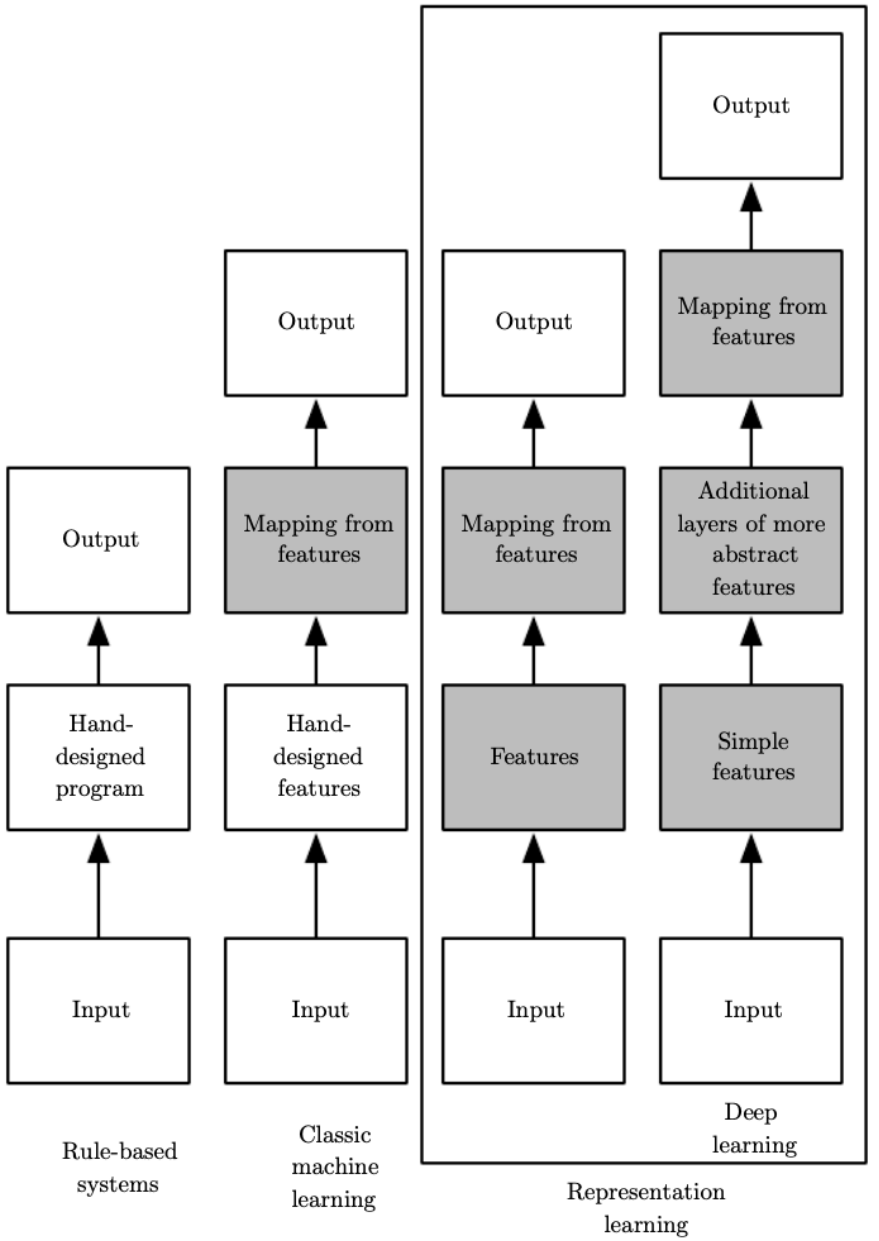


Figure 1.5: Flowcharts showing how the different parts of an AI system relate to each other within different AI disciplines. Shaded boxes indicate components that are able to learn from data.

Connections per Neuron

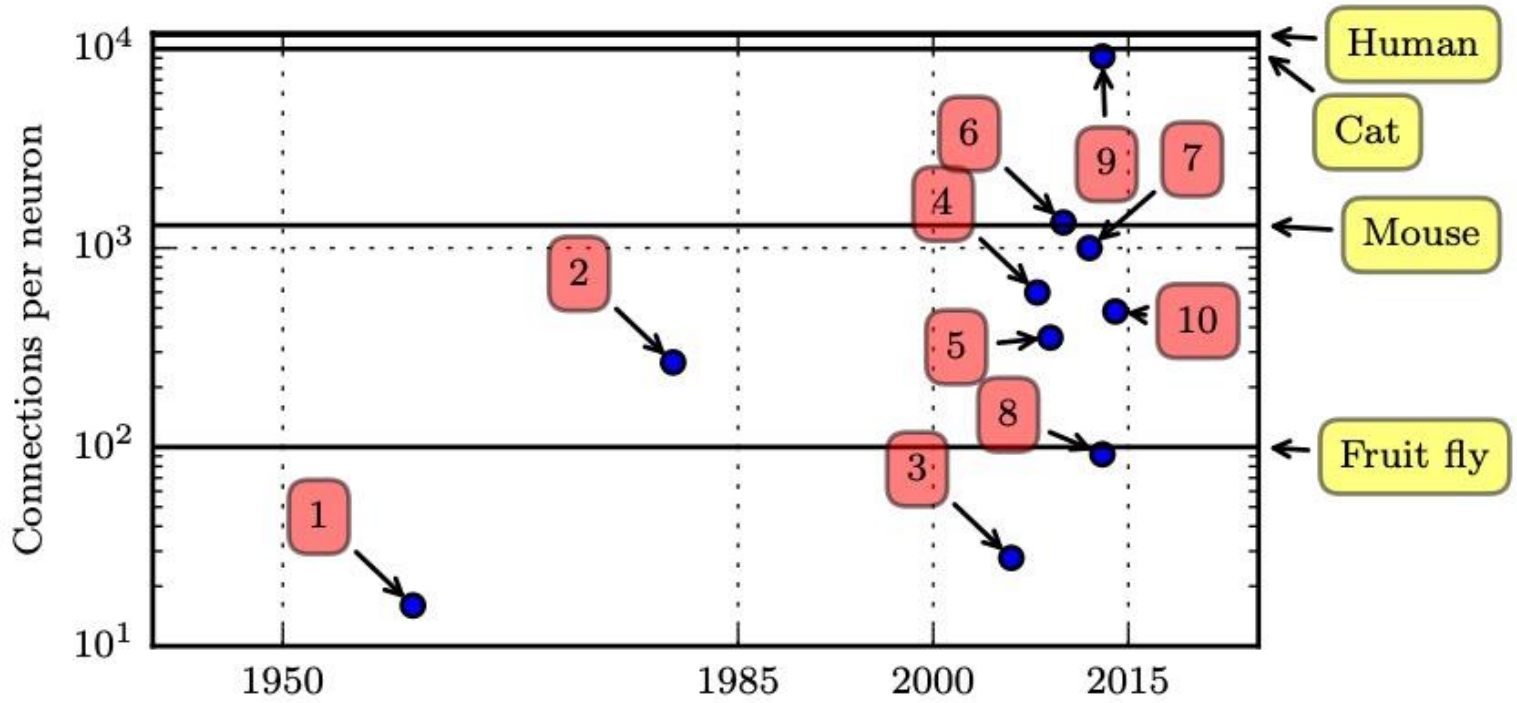


Figure 1.10

Number of Neurons

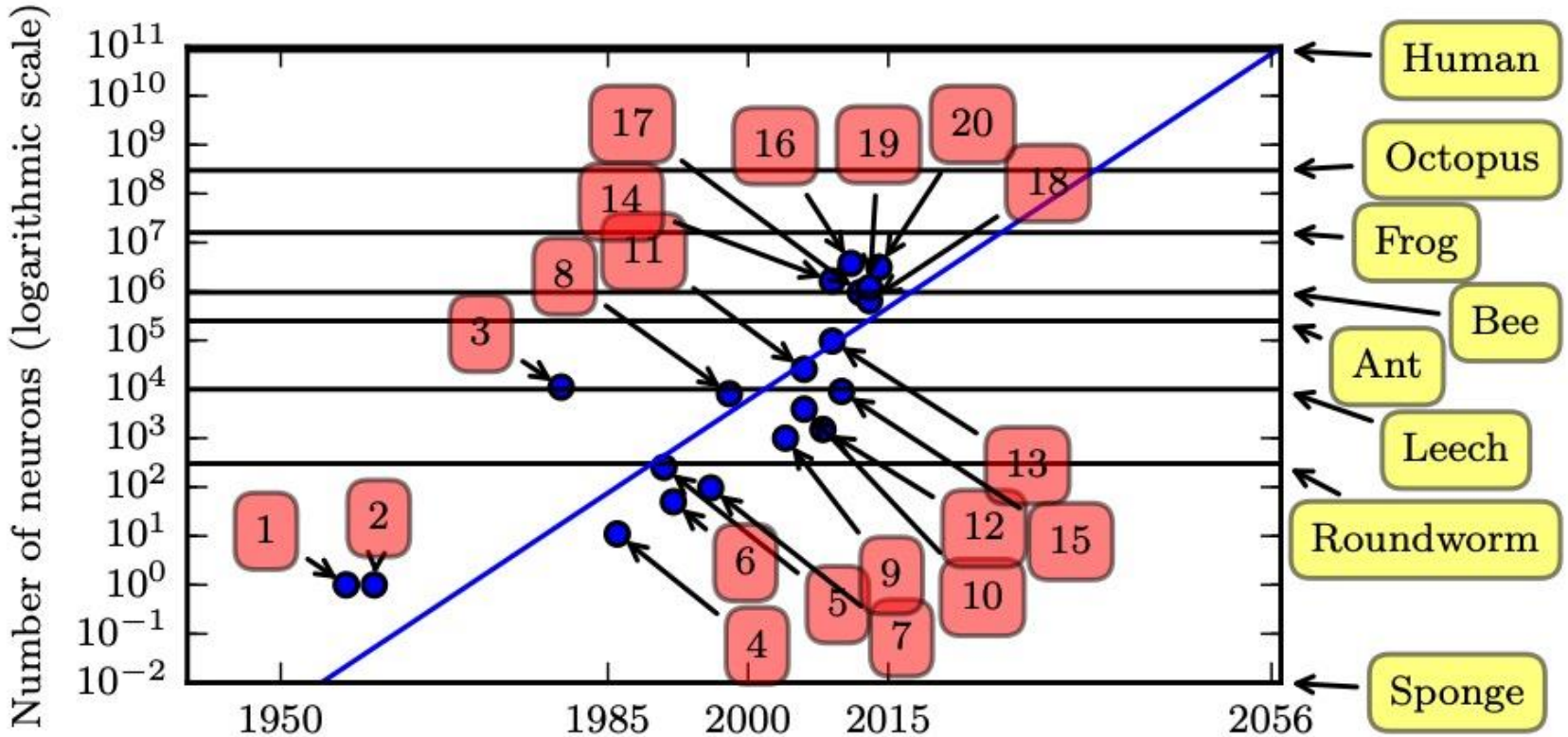


Figure 1.11

(Goodfellow 2016)

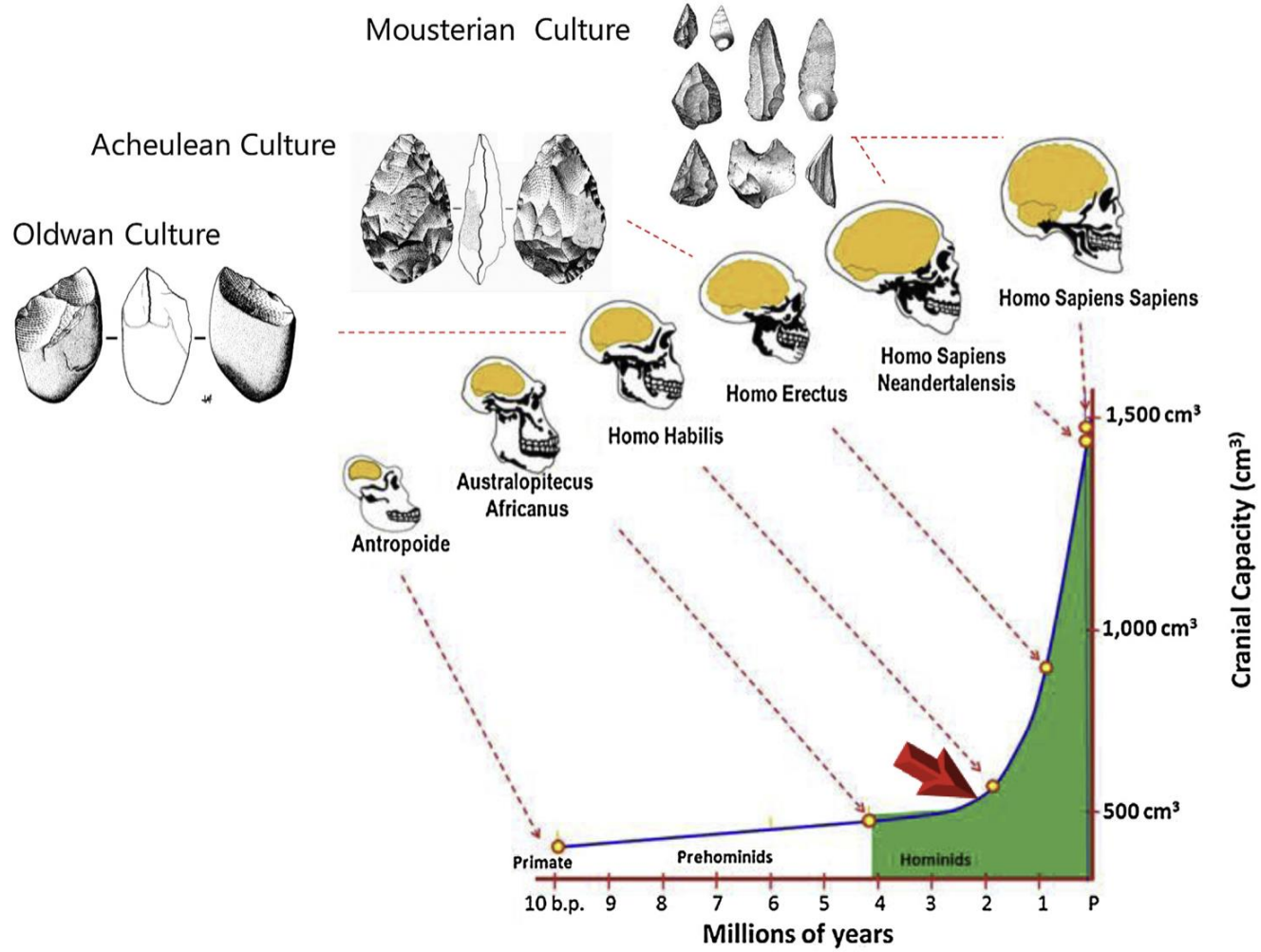
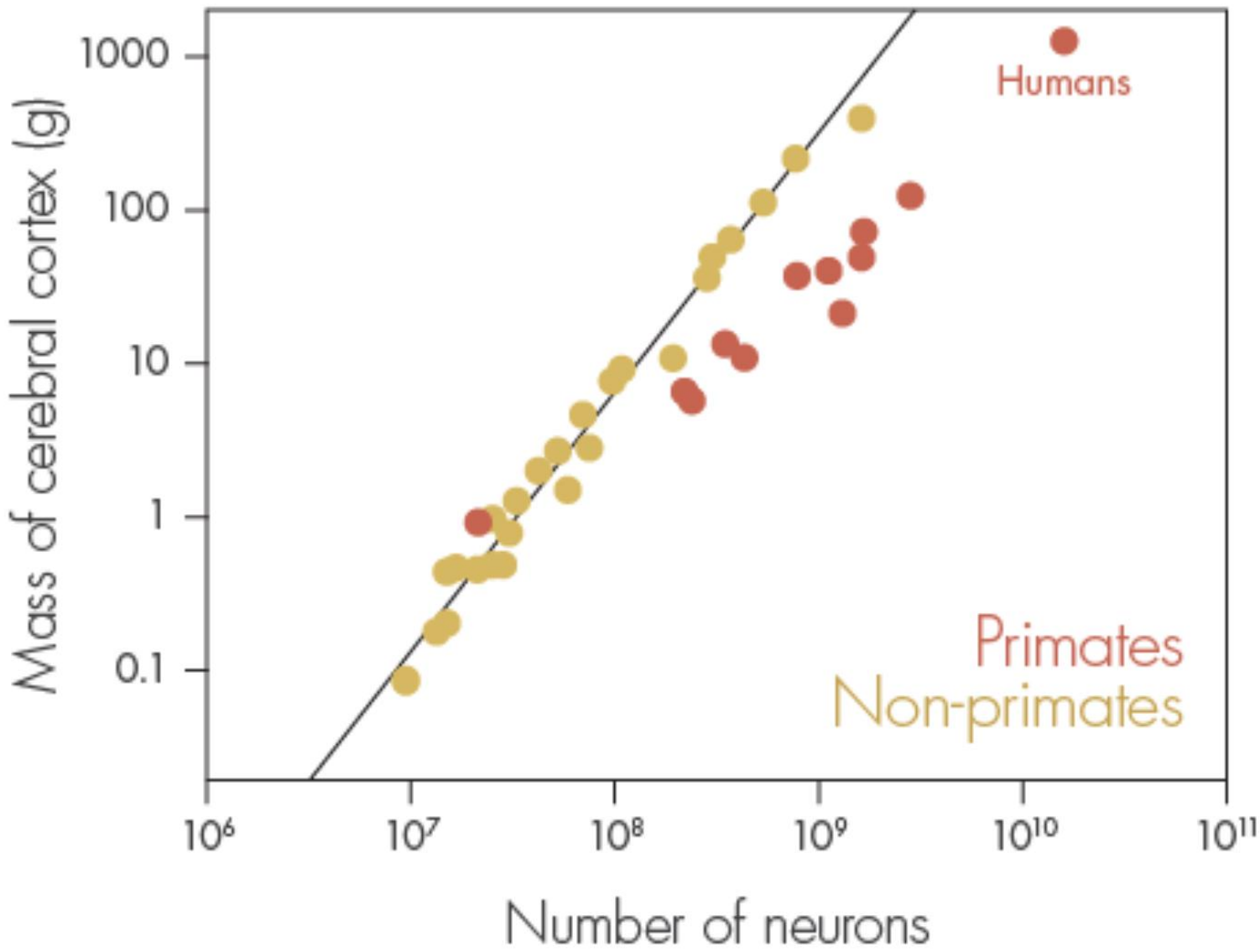


Fig. 1. Transition of cranial capacity (ordinate) along the evolutionary process of various ancestral prehominids and hominids (abscissa) (adapted from "Evolution du volume cérébral des Hominidés, " n.d.). Inset illustrations of a skull and brain depict representative hominids plotted on the graph. Brain expansion suddenly accelerated when Homo Habilis started manufacturing and using stone tools (oblique red arrow). Insets of stone tools (top left) depict the development level of stone tools in each culture. Oldwan culture: The earliest widespread stone tools were simple, usually made with one or a few flakes chipped off with another stone, used by Homo Habilis. Acheulean culture: Stone tools characterized by distinctive oval and pear-shaped "hand-axes," manufactured and used by Homo Erectus. Simple syntax in vocal communication, a primitive form of human language, is thought to have been required for the transmission of this culture. Mousterian culture: Technically complex and significant archaeological industry of lithic tools developed from Homo Sapiens Neanderthalensis throughout early Homo Sapiens Sapiens. Complex and rich semantic contents and syntax, close to modern human language, were necessary for the transmission and inheritance of this culture.

BRAIN DENSITY



Brains and Models

1. Structural lens: synapses and weights

A simple model assigns one independent scalar weight to each synapse.

$P \approx S = N \times k$

N = neurons in the chosen region k = mean incoming synapses per neuron

Regional scope. Use N and k from the same region. For a whole-brain estimate, sum N × k across regions and count each synapse once.

Human example. About 86 billion neurons in the brain, about 16 billion in the cerebral cortex. One neocortical study estimated 1.64×10^{14} synapses. 164 Trillion

Limit. This approximation counts connections. Weight sharing, synaptic dynamics and neuron properties change the relationship to model parameters.

Azevedo et al., 2009; Tang et al., 2001. Full citations in speaker notes.

2. Storage lens: bits per synapse and per weight

Evidence. Bartol et al. inferred at least 26 distinguishable strength levels (about 4.7 bits) in sampled rat hippocampal synapses.

This is a lower-bound estimate under that study's assumptions. Applying it to the human brain requires an explicit extrapolation.

Equivalent weight slots ≈ S × b / q

S = synapses b = assumed bits per synapse q = stored bits per weight

What it measures. A nominal storage comparison at a stated precision.

Limits. Stored bit width differs from learned information. Correlations and unequal state probabilities affect capacity. Other biological states add detail.

Bartol et al., 2015. Full citation and assumptions in speaker notes.

3. Graph lens: a description of the wiring

A simple edge list stores both endpoints and a strength for each connection.

Bits per edge ≈ $2\lceil \log_2 N \rceil + b$

With N = 86 billion neurons: 37 bits per endpoint, or 74 address bits.

If b = 4.7, the idealized total is 78.7 bits per edge.
That equals about 4.9 storage words of 16 bits each.

Encoding matters. Adjacency lists, spatial structure and compression can reduce this description length. It is a property of the chosen encoding.

Meaning. These are storage-word equivalents. Dividing by 16 does not turn connection addresses into additional learned parameters.

Address arithmetic is illustrative. Inputs: Azevedo et al., 2009; Bartol et al., 2015.

4. Dynamics lens: reuse of connections over time

Unrolling a recurrent model reuses the same weights at successive time steps.

Connection evaluations ≈ a × S × T

a = fraction of connections evaluated per step T = number of steps

Illustrative example. $S = 1.6 \times 10^{14}$, $T = 100$ and $a = 1$ give 1.6×10^{16} connection evaluations.

Unique weights. The model still has 1.6×10^{14} synaptic weights under the one-weight-per-synapse assumption.

Caveat. T and a depend on the model. A synaptic event has no universal conversion to FLOPs, and biological activity is asynchronous.

Recurrent weight reuse: PyTorch RNN documentation. Biological scale: Tang et al., 2001.

5. Regional lens: a brain area and a model stage

A regional comparison keeps the biological scope closer to the task.

Scope. Choose a region, such as primary visual cortex (V1),
and define which local and long-range inputs count.

Connections. Estimate $S_r = N_r \times k_r$ using compatible regional data.
An ANN with untied scalar weights can match that connection count.

Function. Test predictions of neural responses or behavior.
Matching counts alone does not establish comparable capability.

Biological detail. A neuron can have nonlinear dendrites and changing internal state. Feedback and plasticity affect the comparison.

Azevedo et al., 2009; Beniaguev et al., 2021; PyTorch Conv2d documentation.

Worked example: human neocortical scale

Use $S \approx 1.6 \times 10^{14}$ synapses, rounded from a neocortical study.
For the storage calculation only, assume b = 4.7 bits per synapse.

Comparison	Calculation	Result
Synaptic weights	$P \approx S$	160 trillion weights
16-bit equivalents	$S \times 4.7 / 16$	47 trillion slots
8-bit equivalents	$S \times 4.7 / 8$	94 trillion slots

The first row counts independent weights in an idealized model.
The other rows compare nominal storage under an extrapolated assumption.

Tang et al., 2001 (human neocortex); Bartol et al., 2015 (rat hippocampus).

Comparisons across mammals

1. Define the population. Use the species, brain region and age group relevant to the question. Record sources and uncertainty.

2. Count consistently. Use regional N and incoming k, or measured synapse density × regional volume. Sum regions only with compatible boundaries.

3. Choose the quantity. Report connections, storage bits, wiring description or connection evaluations. Each quantity has different assumptions.

4. Check sensitivity. Vary uncertain inputs. State weight sharing, precision and time horizon before interpreting the result.

For example, about 71 million neurons is a published whole-mouse-brain estimate. A cortical synapse-per-neuron value needs a cortical neuron count.

Herculano-Houzel et al., 2006. Full citation in speaker notes.

Summary: five ways to compare brains and models

Lens	Quantity	What it tells us
Structure	Synapses S	Count of connections
Storage	S × assumed bits per synapse	Nominal storage budget
Wiring	Address bits + strength bits	Description length
Dynamics	Connection evaluations	Work over a defined interval
Region	A brain area and model stage	A functional hypothesis to test

Parameter count, storage and computation answer different questions.
The biological scope and modeling assumptions determine the comparison.

Summary: the human numbers and their limits

≈86 billion neurons. A whole-brain estimate, with about 16 billion in the cerebral cortex. Regional differences matter.

≈160 trillion synapses. A rounded estimate for the human neocortex from one study, rather than a whole-brain total.

≈47 trillion 16-bit slots. A conditional storage equivalent if every synapse contributes 4.7 bits. That assumption extrapolates from rat hippocampus.

These estimates describe different quantities and populations. They do not establish a human-equivalent AI model size.

Azevedo et al., 2009; Tang et al., 2001; Bartol et al., 2015.

Summary: what makes a useful comparison

A clear question. The relevant brain region and task define the scope.

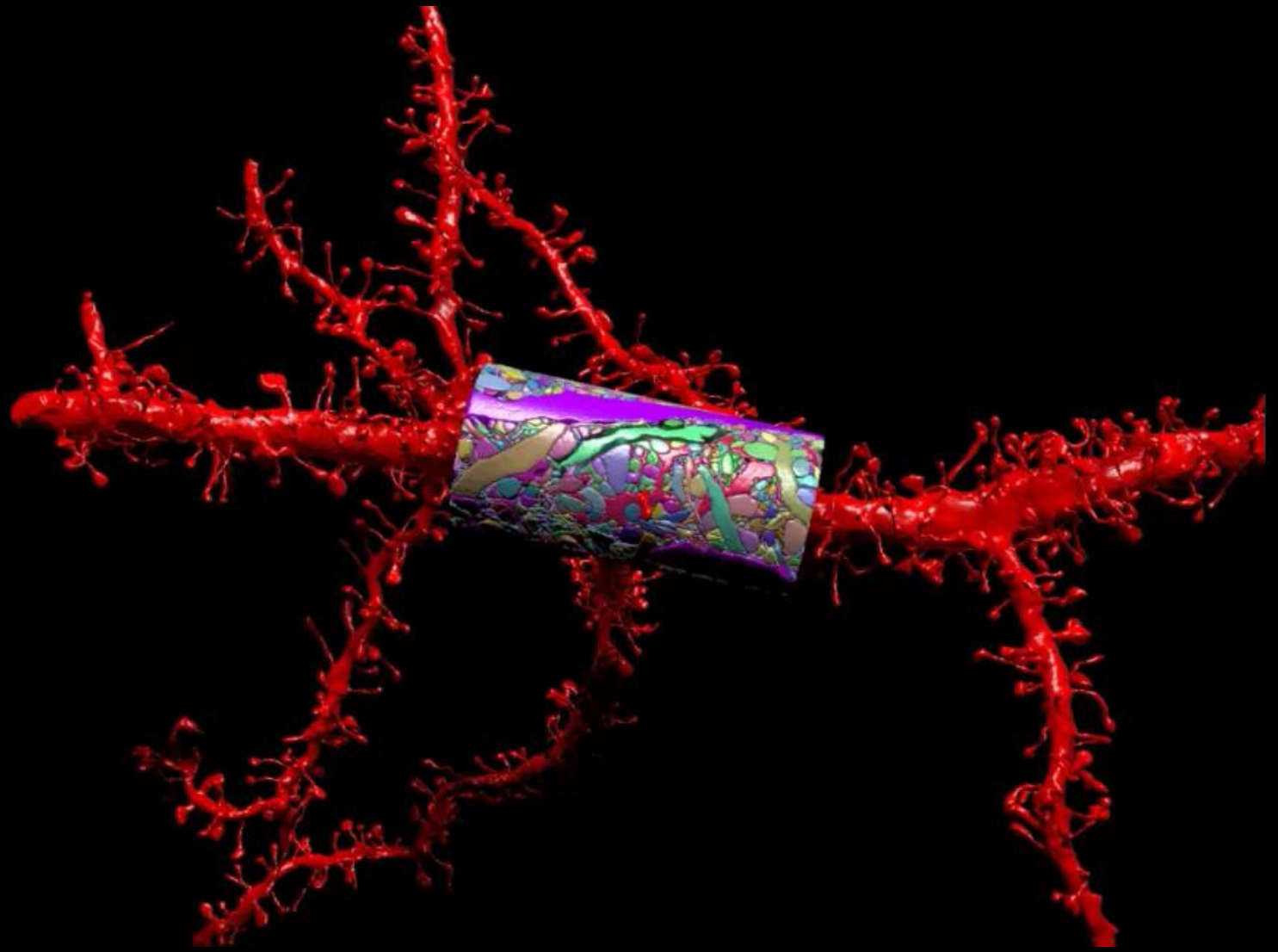
Explicit units. Connections, bits and operations remain separate quantities.

Visible assumptions. Precision, weight sharing and time determine how a biological count maps to a model.

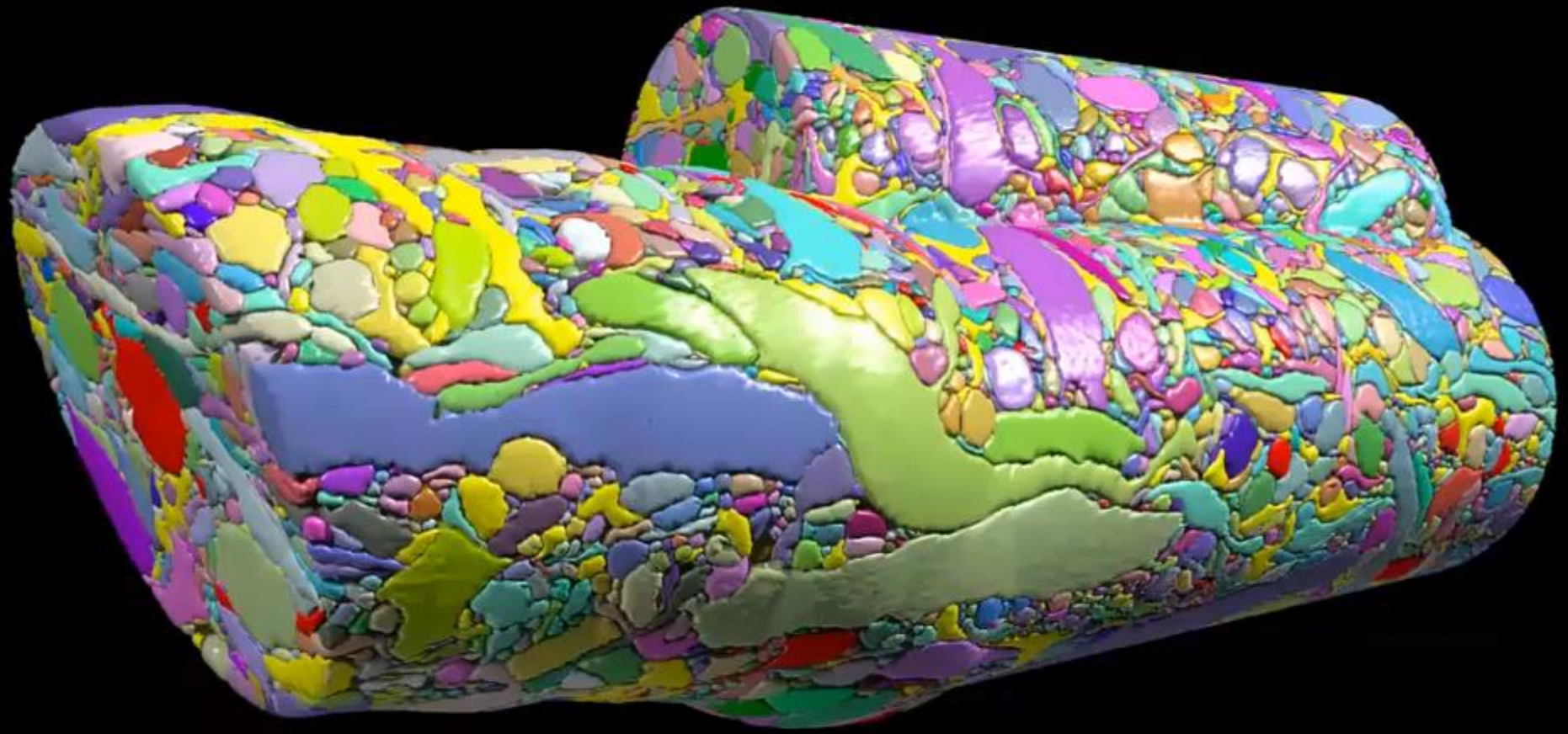
Functional evidence. Neural or behavioral tests are needed to evaluate what a model captures about the brain.

There is no single conversion from synapses to AI capability.

Progress on the (micro) Connectome

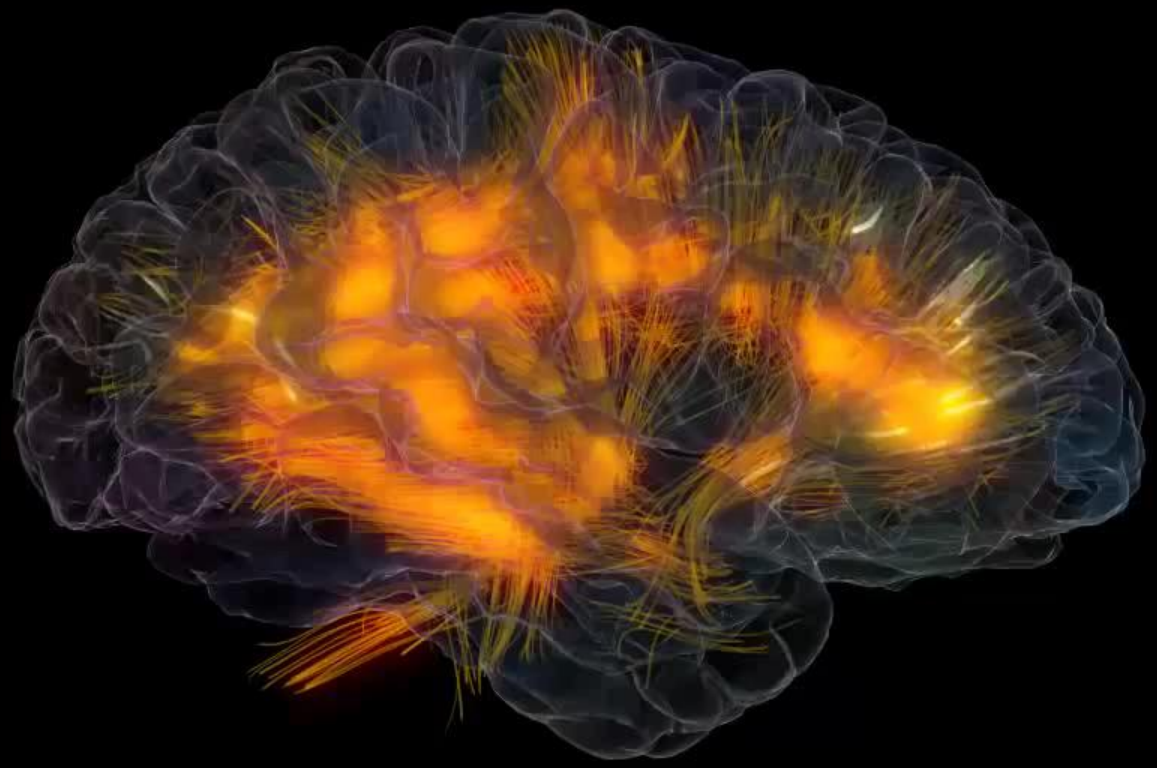


Bobby Kasthuri, et. al. Argonne, Uchicago and Harvard



Kasthuri et al., Cell 2015

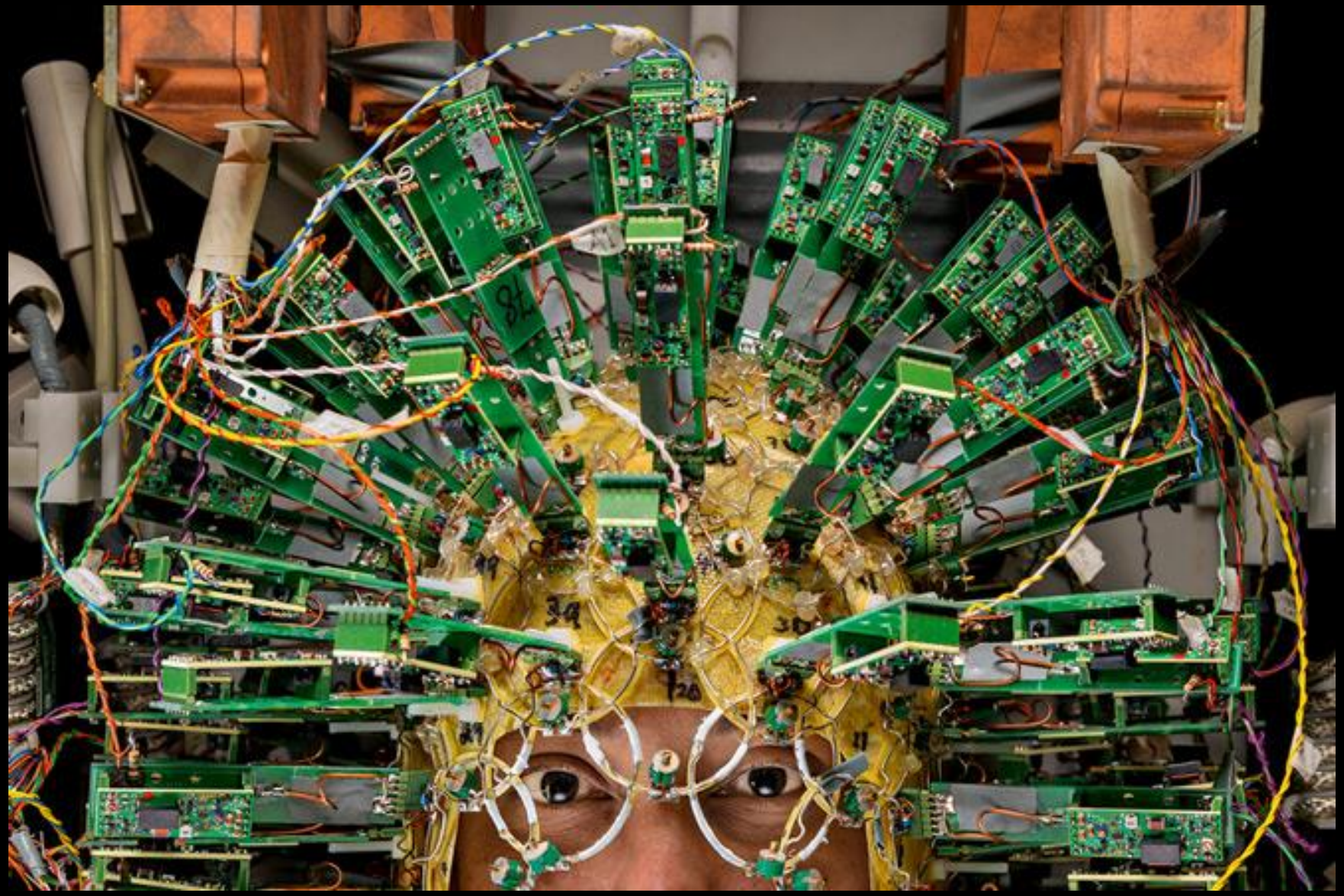
GlassBrain — Visualization



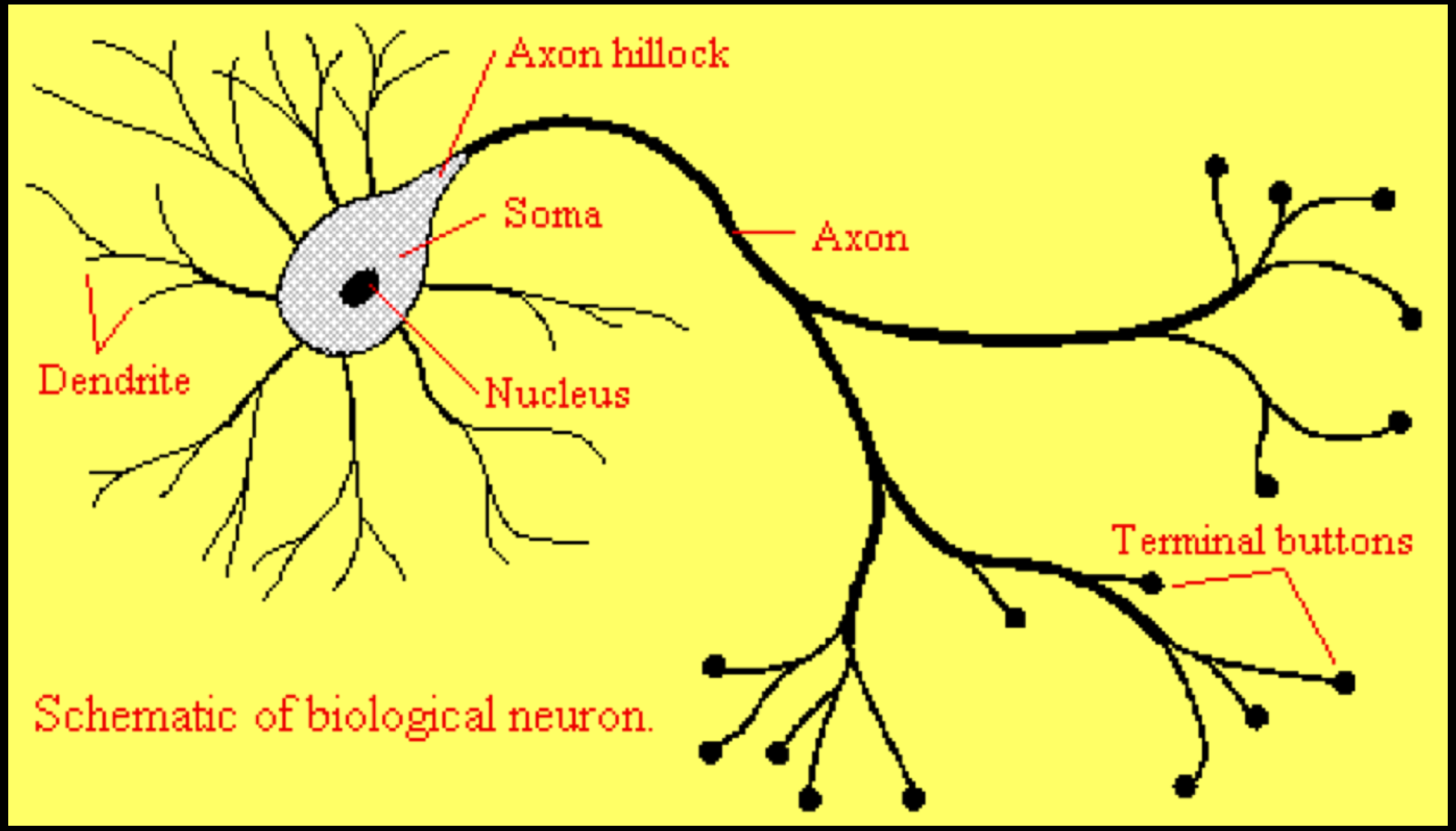
EEG (color indicates “power” and frequency, MRI (Diffusion Tensor Imaging) for structure

Adam Gazzeley Lab - UCSF

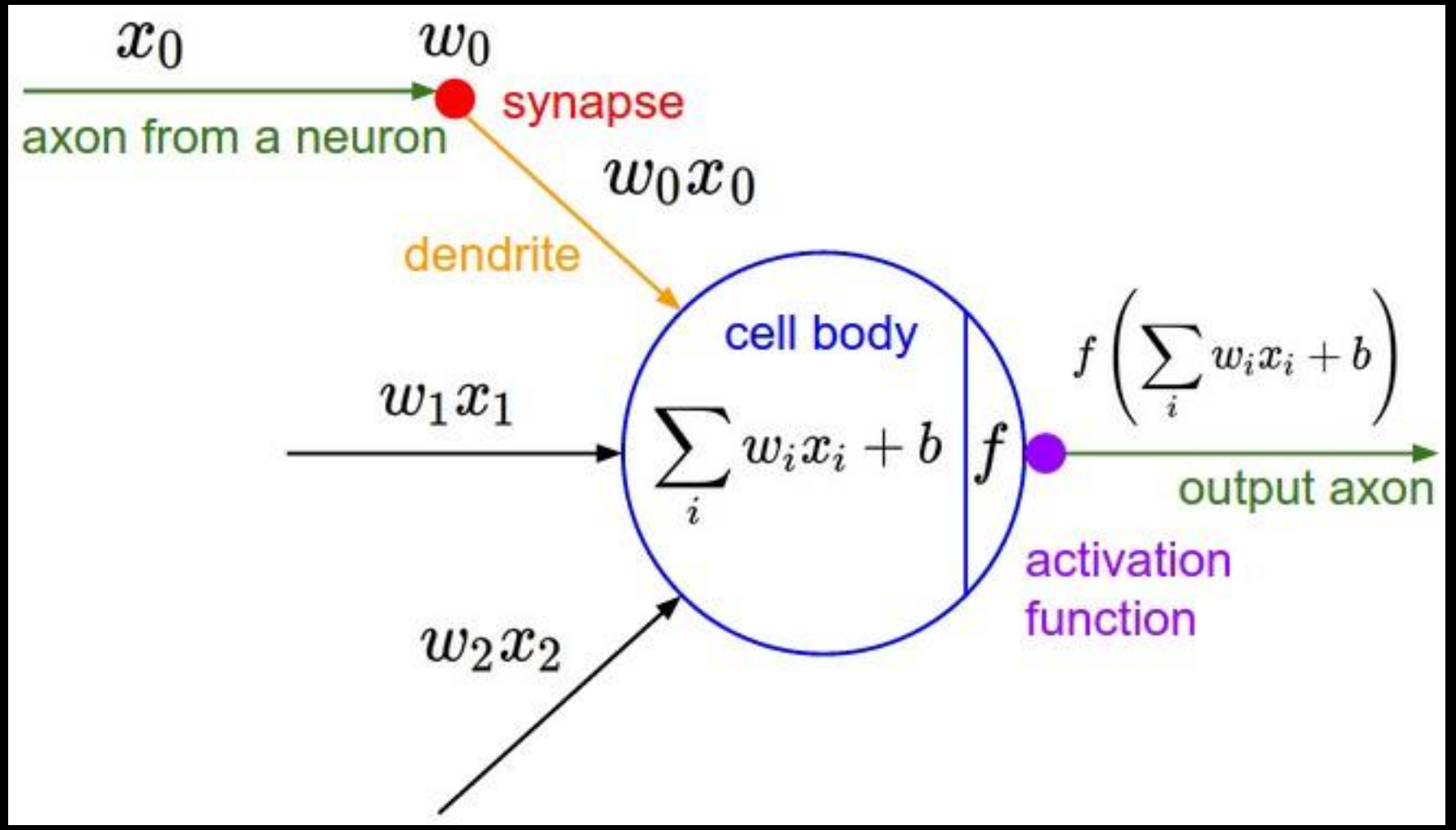
Brain-mapping tools

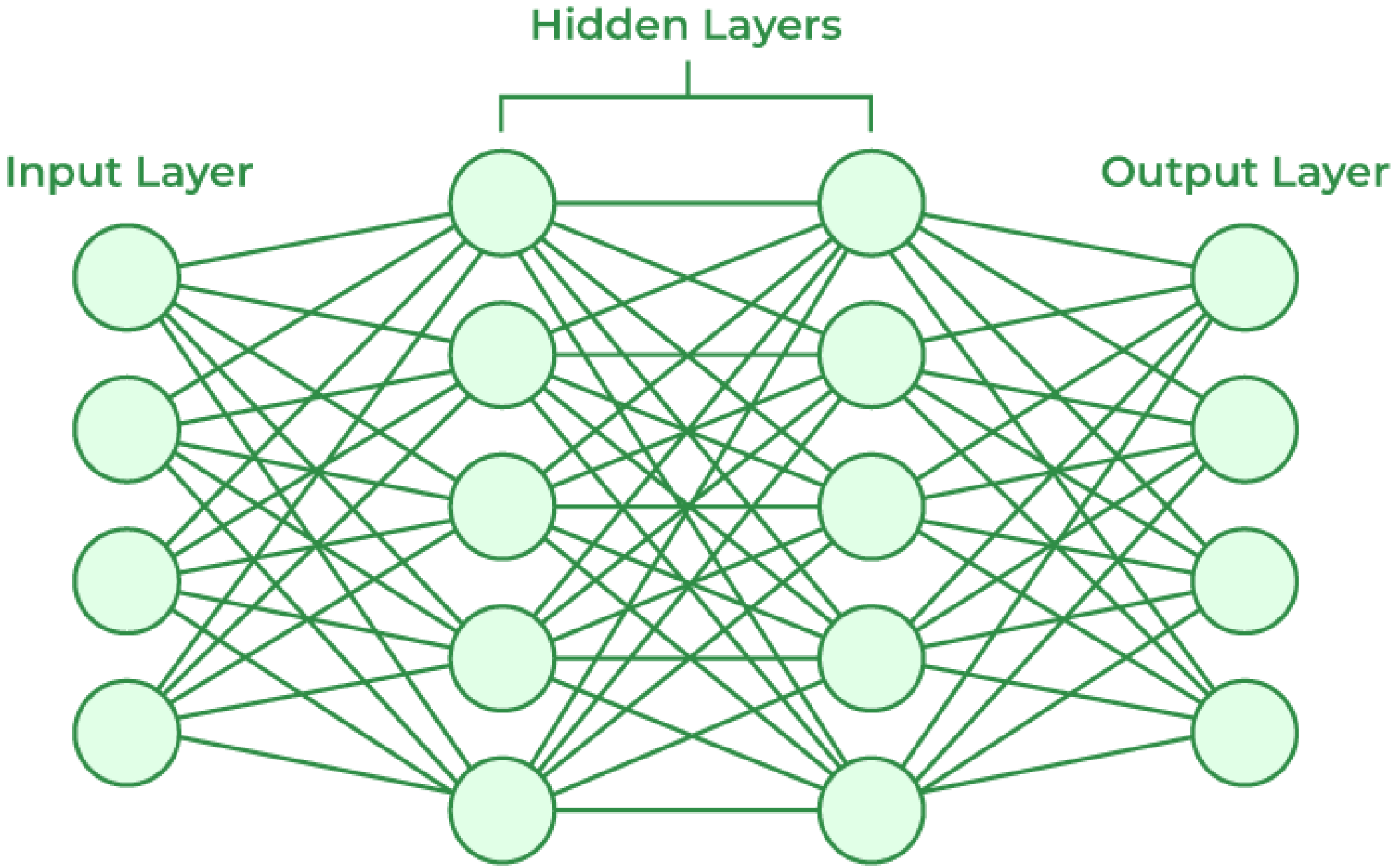


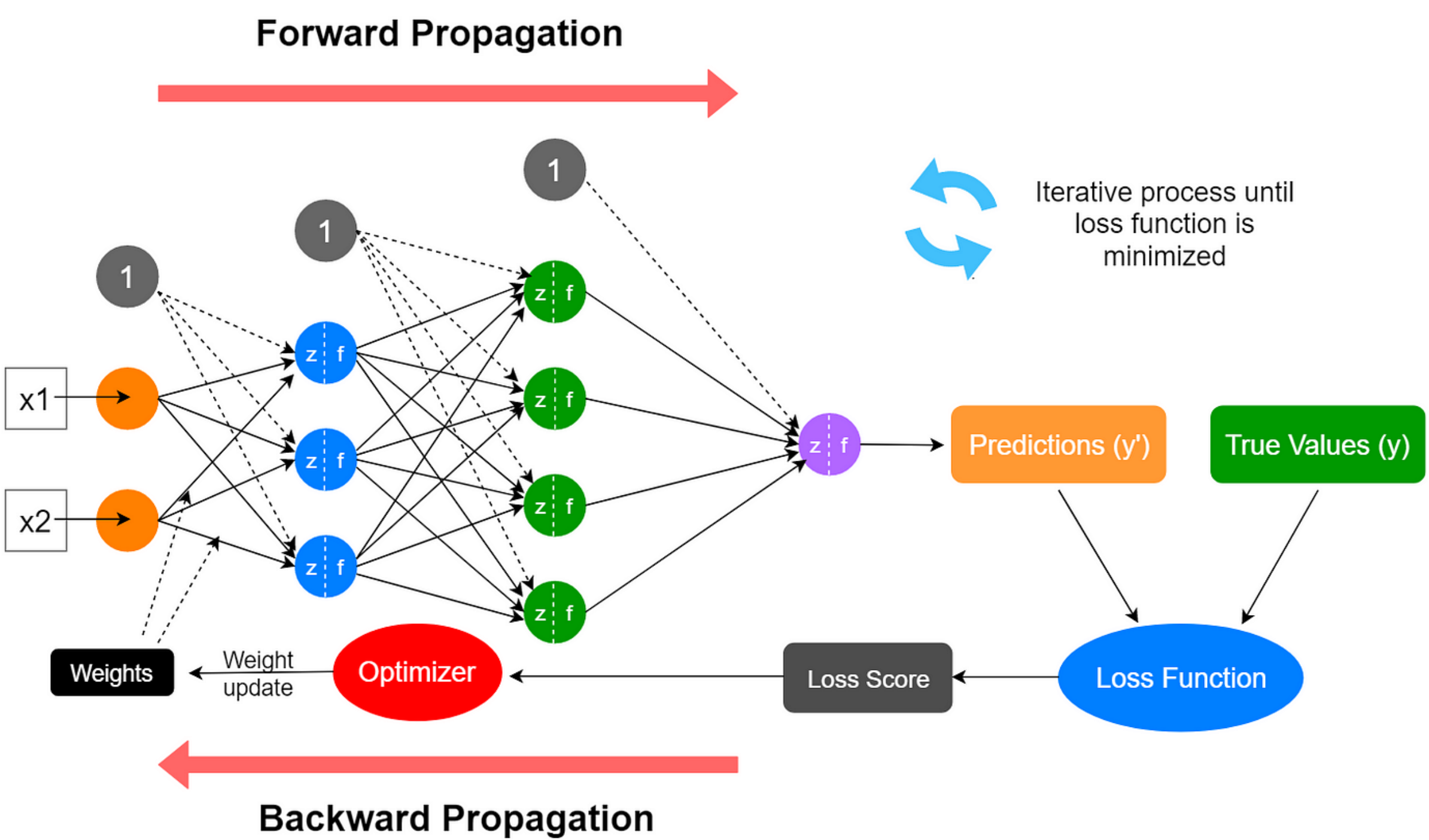
What is a Neuron?



Mathematical Abstraction




Hidden Layers
Input Layer
Output Layer


Forward Propagation
1
1
1
x1
x2
z
f
Iterative process until loss function is minimized
Predictions (y')
True Values (y)
Loss Function
Loss Score
Optimizer
Weight update
Weights
Backward Propagation

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors[2]. Learning becomes more interesting but more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons, there are 'feature analysers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

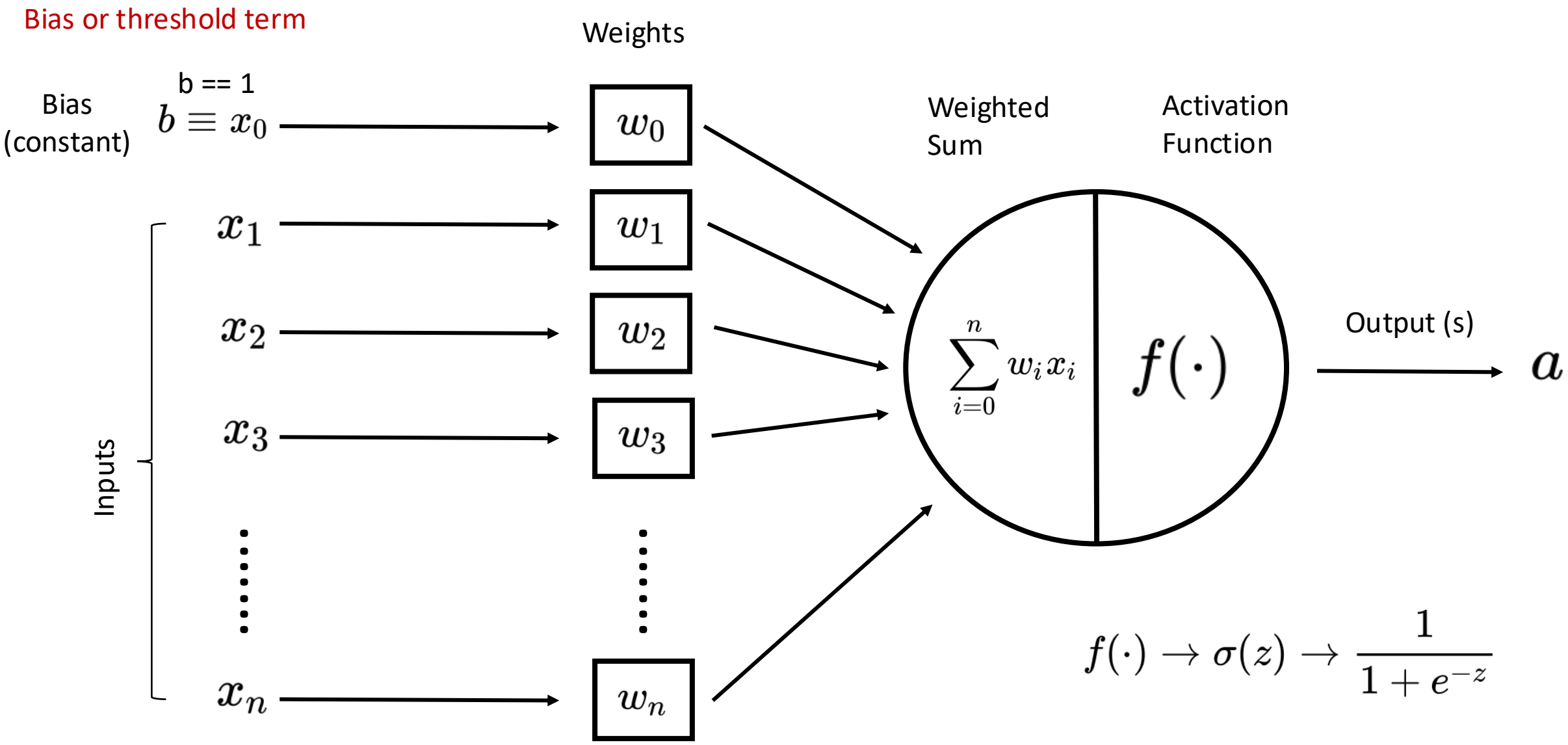
The total input, x_j, to unit j is a linear function of the outputs, y_i, of the units that are connected to j and of the weights, w_{ji}, on these connections

$$x_j = \sum_i y_i w_{ji} \tag{1}$$

Units can be given biases by introducing an extra input to each unit which always has a value of 1. The weight on this extra input is called the bias and is equivalent to a threshold of the opposite sign. It can be treated just like the other weights.

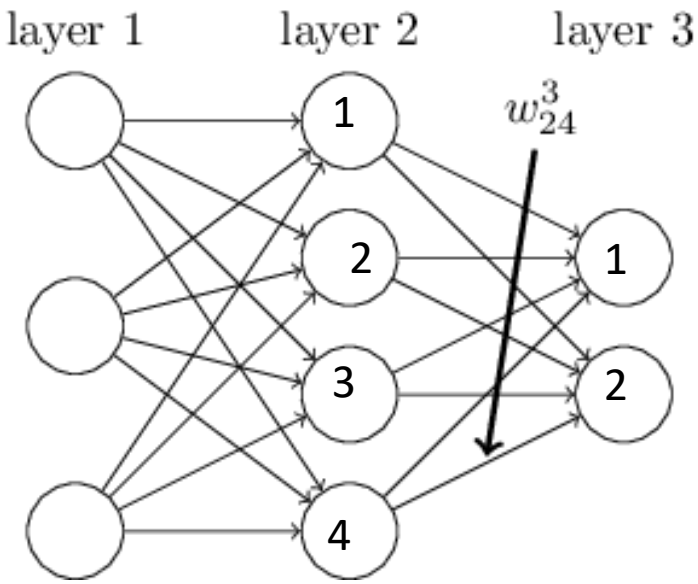
A unit has a real-valued output, y_j, which is a non-linear function of its total input

$$y_j = \frac{1}{1+e^{-x_j}} \tag{2}$$


Bias or threshold term
Weights
b == 1
Bias
(constant)
$b \equiv x_0$
w_0
Weighted
Sum
Activation
Function
x_1
w_1
x_2
w_2
x_3
w_3
Inputs
$\sum_{i=0}^{n} w_i x_i$
$f(\cdot)$
Output (s)
a
x_n
w_n
$f(\cdot) \rightarrow \sigma(z) \rightarrow \frac{1}{1+e^{-z}}$
Example activation function

Convention for naming things

$(l-2)^{th}$ $(l-1)^{th}$ $(l)^{th}$



w^l_{jk} is the weight from the kth neuron in the $(l-1)^{\text{th}}$ layer to the jth neuron in the lth layer

a^l_j

Activation of neuron j in layer l

w^l_{jk}

Weight

l == to (⟶) layer

j == to (⟶) neuron

k == from (⟵) neuron

With these notations, the activation a^l_j of the jth neuron in the lth layer is related to the activations in the $(l-1)^{th}$ layer by the equation

$$a^l_j = \sigma\left(\sum_k w^l_{jk} a^{(l-1)}_k + b^l_j\right)$$

To rewrite this expression in a matrix form we define a *weight matrix* w^l for each layer, l. The entries of the weight matrix w^l are just the weights connecting to the lth layer of neurons, that is, the entry in the jth row and k^{th} column is w^l_{jk} . We can simplify our vector notation a bit by dropping the explicit bias term.

$$a^l = \sigma\left(w^l a^{(l-1)} + b^l\right) \rightarrow a^l = \sigma\left(w^l a^{(l-1)}\right)$$

Note that *bias is also known as threshold*, which will now make more sense to you.

Say bank approves credit if $(\sum_{i=1}^{d} WiXi > threshold)$ and deny if it is lesser. So this linear formula can be written as a hypothesis which is:

$$h(x) = (\sum_{i=1}^{d} WiXi - threshold)$$

Suppose $threshold = -W0$, above equation can be rewritten as

$$h(x) = sign((\sum_{i=1}^{d} WiXi) + W0)$$

Introducing X0=1 in the equation.

$$h(x) = sign((\sum_{i=1}^{d} WiXi) + W0X0)$$

Now we can simply write the hypothesis equation as

$$h(x) = sign(\sum_{i=0}^{d} WiXi).$$

This is the standard form. The vector form is $h(x) = sign(W'X)$.